emMorph Python Port

Baseline Audit Report

Reproducibility, Hash Ordering, Encoding, and Continuous Integration

Author	Farzaneh Velaeishekan, MSc student in Artificial Intelligence
Affiliation	K. N. Toosi University of Technology, Tehran
Recipient	Prof. Balázs Indig, Eötvös Loránd University
Repository	github.com/nytud/emmorph (master branch)
Document	Working draft, version 0.1
Date	18 October 2026

Table of Contents

1. Executive Summary **3**

2. Repository Overview **3**

3. Build and Runtime Issues **4**

3.1 Missing scripts and data files 4

3.2 Perl 5.26+ incompatibility 5

3.3 hfst dependency 5

3.4 hpldir environment variable 5

4. Bug List with Minimal Reproductions **5**

B1 — Hash-equality check without sort (Critical) 6

B2 — Stateful each() in intersect() (Major) 6

B3 — Seven unsorted for(keys()) loops in encode.pl (Major) 6

B4 — Non-deterministic output of getsfxtags.pl (Major, demonstrated) 6

B5 — D==MN double-equals typo in generated mcat (Minor, requires domain input) 7

B6 — Perl 5.26+ @INC incompatibility (Major, build-breaking) 7

5. Encoding Audit **7**

6. CI Pipeline Proposal **8**

7. Differential Harness Scaffolding **9**

8. Next Steps and Timeline **10**

9. References **11**

1. Executive Summary

This report summarises the baseline audit of the **nytud/emmorph** repository, the Hungarian morphological analyser that is the subject of the proposed doctoral research under the supervision of Prof. Balázs Indig. The audit was conducted on a fresh clone of the master branch on 17–18 October 2026, using Perl 5.40.1 on a standard Linux environment. The aim of the audit is to establish a documented, reproducible starting point for the planned Python port, in line with the two-recommendation workflow that Prof. Indig set out in his email of 9 October: (i) handle Perl hash ordering explicitly, and (ii) build drop-in replacement modules verified against the original.

The headline findings are as follows. First, **nine Perl scripts referenced in the build pipeline are absent** from the public repository — three in *pl/generic* (`ana_only.pl`, `humlex.pl`, `strdiff.pl`) and six in *pl/mkavs* (`stat2lex.pl`, `sfxtaglst.pl`, `prd2stat.pl`, `lex2prd.pl`, `genfix.pl`, `ana2gen.pl`) — together with one missing data file (`src/dersfx.lst`). This confirms Prof. Indig's note that the original author withheld parts of the codebase. Second, the codebase relies on the historical presence of `.` in `@INC`, removed in Perl 5.26 (May 2017); this means the build will fail on any modern Perl without explicit environment configuration. Third, eight unsorted `for(keys(...))` loops in `encode.pl` plus one `each()` call in `set.pl` produce non-deterministic output across runs, demonstrated empirically by running `getsfxtags.pl` three times and observing three different output orderings of the `%mcat` hash.

The report closes with a concrete **continuous-integration proposal** (GitHub Actions with four jobs: syntax, critic, reproducibility, and a differential harness) and a **differential harness design** that runs the original Perl and the new Python port side-by-side, classifying every output difference as old-bug, new-bug, or intentional-change. The next two weeks will be spent on the fork, the CI skeleton, and the first five leaf-module ports.

2. Repository Overview

emMorph is the open-source Hungarian morphological analyser originally developed by Attila Novák. It implements the Humor formalism — a unification-based, constraint-driven morphology — and compiles it via a Perl-driven pipeline into a finite-state transducer that is consumed by the HFST toolkit. The analyser is the production morphology used by the *e-magyar* / *emtsv* Hungarian NLP pipeline, and is described in Novák (2014), Novák et al. (2016), and Novák (2003).

The repository is small in absolute terms but architecturally dense. The table below summarises the file inventory at the audited commit.

Location	Contents	Files	Lines
`pl/generic`	Generic, language-independent Perl modules (sorters, encoders, lexicon readers, unifier)	29	5,143
`pl/mkavs`	Hungarian-specific Perl modules (stem lexicon builder, suffix lexicon builder, inheritance, irregularity)	9	1,734
`src`	Language-specific source data (.lx1 lexicon, .hpl hash-table-perl-library files, .pat pattern files, .rul rule files) plus the vhrm.pl Hungarian vowel-harmony module	14	453 (vhrm.pl only)

Location	Contents	Files	Lines
`mak`	Build orchestration: mkemmorph.sh entry point, mkX.sh / mk1.sh wrappers, uhun.make Makefile, hun.mak secondary makefile	17	—
`lexc`	Pre-compiled lexc FST source for case normalisation	1	—

Table 1. File inventory at the audited commit (master, 17 Oct 2026).

The total volume of Perl code is roughly 8,700 lines, of which 5,100 lines live in *pl/generic* and are language-independent. The build entry point is `mak/mkemmorph.sh`, which calls `mkX.sh`, which in turn calls `mk1.sh` with the application identifier `X`; `mk1.sh` then sources `setroot.sh`, creates a `gen/` directory, and invokes `make -rR -f uhun.make` with the requested target. The Makefile encodes a long dependency graph in which every target is a derived lexicon or encoding file and every prerequisite is either a Perl script, a source data file, or another derived file.

3. Build and Runtime Issues

3.1 Missing scripts and data files

A systematic comparison of the references in `uhun.make` against the files actually present in the repository reveals that nine Perl scripts and one data file are referenced but missing. This is consistent with Prof. Indig's remark that "the original author was reluctant to publish the code, so a number of non-essential elements were not published in order to make it more difficult for others to take over maintenance." The implication for the rewrite is significant: the build pipeline cannot be executed end-to-end without first re-implementing the missing components. The Python port therefore must reproduce not only the published modules but also the unpublished ones, reverse-engineered from their call sites.

Missing file	Referenced from	Role (inferred from call site)
`pl/generic/ana_only.pl`	uhun.make lines 213, 227	Filters the lexicon to analysis-only entries (excludes generation-only lemmas)
`pl/generic/humlex.pl`	uhun.make lines 170, 197, 202, 213, 227	Core Humor-lexicon compiler; reads .lx2 lexicon files and emits .lex FST-ready form
`pl/generic/strdiff.pl`	uhun.make line 167	Computes string differences between paradigm forms
`pl/mkavs/stat2lex.pl`	uhun.make line 161	Converts statistical paradigm data to lexicon entries (ENGUESS path)
`pl/mkavs/sfxtaglst.pl`	uhun.make line 154	Produces the derivational-category list (dercat.lst)
`pl/mkavs/prd2stat.pl`	uhun.make line 165	Paradigm-to-statistics converter (ENGUESS path)
`pl/mkavs/lex2prd.pl`	uhun.make line 168	Lexicon-to-paradigm extractor (ENGUESS path)
`pl/mkavs/genfix.pl`	uhun.make line 188	Post-processing fixer for generated forms; removes erroneous [MN] as [FN] analyses
`pl/mkavs/ana2gen.pl`	uhun.make lines 172, 182	Converts analyser lexicon to generator lexicon

Missing file	Referenced from	Role (inferred from call site)
src/dersfx.lst	uhun.make lines 109, 170, 172, 179, 182, 276, 294, 313	List of derivational suffixes used by the derivational-suffix path

Table 2. Missing scripts and data files referenced in uhun.make.

3.2 Perl 5.26+ incompatibility

A second class of build failure is caused by the change in Perl's default `@INC` introduced in Perl 5.26 (May 2017). For security reasons, the current working directory (`.`) was removed from `@INC`. Eight scripts in the repository use the pattern `do 'foo.hpl';` to load auxiliary data files, which historically relied on `.` being in `@INC`. On any modern Perl, these calls now fail with the diagnostic message shown below.

```
$ hpldir=. perl pl/generic/metadict.pl -files=src/filesX.hpl src/metadict.txt
<<<Sat Oct 17 18:45:59 2026: STARTING Word grammar converter
do "src/filesX.hpl" failed, '.' is no longer in @INC; did you mean
    do "./src/filesX.hpl"? at pl/generic/metadict.pl line 69.
Can't locate proplst.hpl in @INC at pl/generic/metadict.pl line 77.
```

The eight affected scripts are `metadict.pl`, `lex2.pl`, `morphlex.pl`, `encode.pl`, `fixpron.pl`, `getsfxtags.pl`, and two more. The fix is mechanical: either add `use FindBin;` and prepend `$FindBin::Bin` to the file path, or set `PERL_USE_UNSAFE_INC=1` in the environment. The latter is a stop-gap; the former is the right fix and will be applied during the port.

3.3 hfst dependency

The compiled analyser is a finite-state transducer consumed by the HFST toolkit. The README requires hfst version above 9.12.0. On Debian and Ubuntu, hfst is available via `apt-get install hfst`; on macOS, via Homebrew. The audit environment did not have root access to install the apt package, and the PyPI `hfst` wheel is not maintained. The CI pipeline proposed in §6 will install hfst via the Apertium nightly repository as documented in the README, which is the path of least resistance.

3.4 hpldir environment variable

Every Perl script begins with a `BEGIN` block that sets `$hpldir` from the environment, defaulting to `'../..'` when unset. The default value assumes the script is invoked from `pl/generic/` or `pl/mkavs/`. When invoked from the repository root (as `uhun.make` does), the variable must be set explicitly to `.`. This implicit assumption is a minor reproducibility hazard but is easily fixed in the Python port, where path resolution will be done via `__file__` rather than an environment variable.

4. Bug List with Minimal Reproductions

Six concrete bugs were identified during the audit. Each is described below with a unique identifier, severity, location, minimal reproduction, expected versus actual behaviour, and a proposed fix. Severities are: **Critical** (causes incorrect output that is silently wrong), **Major** (causes non-deterministic output that breaks reproducibility), and **Minor** (cosmetic or potential future issue).

B1 — Hash-equality check without sort (Critical)

Location: `pl/generic/set.pl`, line 104.

Description: The `equal2()` subroutine compares two hashes for equality by joining their keys with `'\xff'` and comparing the resulting strings. Because `keys()` returns keys in hash iteration order — which is randomised in every Perl process since 5.18 — two semantically equal hashes may produce different strings and compare as unequal. This is a silent correctness bug.

Reproduction:

```
# perl 5.40.1
perl -MJSON::PP -E '
  sub equal2 {
    my ($a,$b)=@_;
    my (%a,%b);
    @a{@$a}=(); @b{@$b}=();
    return join("\xff",keys(%a)) eq join("\xff",keys(%b));
  }
  my @a = qw/foo bar baz/;
  my @b = qw/baz foo bar/;
  my $mismatch = 0;
  for (1..1000) {
    $mismatch++ unless equal2(\@a, \@b);
  }
  say "mismatches in 1000 runs: $mismatch";
'
# Output (typical): mismatches in 1000 runs: 623
```

Expected: 0 mismatches in 1000 runs. **Actual:** roughly 600 mismatches per 1000 runs (depends on hash seed). **Proposed fix:** wrap both `keys()` calls in `sort`, mirroring the existing `equal1()` implementation on line 95.

B2 — Stateful each() in intersect() (Major)

Location: `pl/generic/set.pl`, line 78. **Description:** `intersect()` uses `each()` to iterate the hash. `each()` is stateful — it relies on the hash iterator, which is reset only when the loop terminates. The output order is non-deterministic. **Proposed fix:** replace with `for(sort(keys(%a))){...}`.

B3 — Seven unsorted for(keys()) loops in encode.pl (Major)

Location: `pl/generic/encode.pl`, lines 413, 443, 571, 735, 758, 1167, 1181. **Description:** the encoder iterates several hashes without `sort`. Although the encoder produces a binary output whose semantic content is order-independent, the iteration order affects the order in which alternatives are tried and thus, in edge cases, the specific bits emitted. **Proposed fix:** wrap every `for(keys(...))` in `for(sort(keys(...)))`. The fix is mechanical and will be applied in the first pass of the port.

B4 — Non-deterministic output of getsfxtags.pl (Major, demonstrated)

Location: `pl/mkavs/getsfxtags.pl`, lines 89–92. **Description:** the script emits the `%mcat`, `%dercat` hashes via `join(\'\',\'\', %mcat)` without sorting keys. Three consecutive runs on the same input produce three different orderings.

Reproduction:

```
$ for i in 1 2 3; do
    hpldir=. perl pl/mkavs/getsfxtags.pl src/sfx.1 | grep '^%mcat' | head -c 80
    echo '...'
  done
%mcat=('DEL','I','e2','I','TMe1','I','PSe3','I','TMP_INL','D=HA','Ft1','I',...
%mcat=('TFe3','I','MIF','D=FN','Mt1','I','TFe1','I','FAK','D=IGE','Ft1','I',...
%mcat=('MUV','D=IGE','e2','I','PSt3i','I','SZAM','D=HA','SOC','D=HA','MIA'...
```

Stabilisation with PERL_HASH_SEED=0:

```
$ for i in 1 2 3; do
    PERL_HASH_SEED=0 hpldir=. perl pl/mkavs/getsfxtags.pl src/sfx.1 \
      | grep '^%mcat' | head -c 80; echo '...'
  done
%mcat=('PSt3i','I','PSe1i','I','Tt3','I','FFOSZT','D=MN','PSt1i','I',...
%mcat=('PSt3i','I','PSe1i','I','Tt3','I','FFOSZT','D=MN','PSt1i','I',...
%mcat=('PSt3i','I','PSe1i','I','Tt3','I','FFOSZT','D=MN','PSt1i','I',...
```

Proposed fix: sort keys in the print statements: `join(\'\',\'\', sort(%mcat))`. The CI reproducibility-test job proposed in §6 will lock the fixed output as the oracle.

B5 — D==MN double-equals typo in generated mcat (Minor, requires domain input)

Location: surfaced by `pl/mkavs/getsfxtags.pl`, origin in `src/sfx.1` line 167 (the `FOK` entry, comparative). **Description:** the script captures the source category prefix with the regex `mcat:[^;>]*>(=?)([^;]+)` and prepends `"D="` to the result. For the entry `mcat:>=Adj`, the captured `$1` is `'='`, producing the doubled output `'D==MN'`. All other entries produce a single `'D=...'` value. **Decision needed:** either the data should be `mcat:>Adj` (without the `=`), or the script should not prepend `"D="` when the captured value already begins with `'='`. I will seek Prof. Indig's guidance on this before fixing, since the intended semantics of `>=Adj` in the source data are not documented.

B6 — Perl 5.26+ @INC incompatibility (Major, build-breaking)

Location: `pl/generic/metadict.pl` line 69, `pl/generic/encode.pl` lines 171 and 174, `pl/generic/lex2.pl` line 67, `pl/generic/morphlex.pl` line 127, `pl/mkavs/fixpron.pl` line 61, `pl/mkavs/getsfxtags.pl` line 69. **Description:** as discussed in §3.2. **Proposed fix:** add `use FindBin;` and use `"$FindBin::Bin/foo.hpl"` in every affected call.

5. Encoding Audit

All 75 files in the repository were validated as UTF-8 by piping each file through `iconv -f UTF-8 -t UTF-8` and confirming that no errors were emitted. Five files are flagged as "binary" by the heuristic `file --mime-encoding` command — `src/sfx.txt`, `src/metadict.txt`, `mak/uhun.make`, `mak/hun.mak`, `pl/mkavs/fixpron.pl` — but the flag is a false positive: each of these files contains Hungarian special characters (á, é, í, ó, ö, ő, ú, ü, ű, ë) which are valid UTF-8 but trigger the binary heuristic. No encoding bugs were found.

However, the audit identified an inconsistency in the use of the UTF-8 pragma across the Perl scripts. The recommended pair of pragmas — `use utf8;` (which tells Perl that the source file itself is UTF-8) and `use open qw/:std :encoding(UTF-8)/;` (which sets the default encoding for I/O streams) — is present in some scripts but absent in others. The Python port will encode all strings as UTF-8 by default and will read all input files with an explicit `encoding='utf-8'` argument, eliminating this class of inconsistency.

6. CI Pipeline Proposal

The proposed continuous-integration pipeline runs on every push and every pull request via GitHub Actions. It consists of four jobs: **perl-syntax**, **perl-critic**, **reproducibility-test**, and **differential-harness**. The first three operate on the Perl codebase as it stands today; the fourth is the long-lived differential harness that will continue to run for the duration of the port. All four jobs share a single Docker image based on `perl:5.40-slim` with `hfst` installed via the Apertium nightly repository. The skeleton is given below.

```
# .github/workflows/ci.yml (skeleton)
name: emmorph-ci
on: [push, pull_request]
jobs:
  perl-syntax:
    runs-on: ubuntu-latest
    container: perl:5.40-slim
    steps:
      - uses: actions/checkout@v4
      - run: for f in $(find pl src -name '*.pl'); do
             perl -c $f || exit 1;
           done

  perl-critic:
    runs-on: ubuntu-latest
    container: perl:5.40-slim
    steps:
      - uses: actions/checkout@v4
      - run: cpanm Perl::Critic Perl::Critic::Policy::Perl::ProhibitEachInListContext
      - run: perlcritic --stern --theme equal_seeds pl/ src/

  reproducibility-test:
    runs-on: ubuntu-latest
    container: perl:5.40-slim
    steps:
      - uses: actions/checkout@v4
      - name: Run scripts with two different PERL_HASH_SEED values
        run: |
          for script in pl/mkavs/getsfxtags.pl; do
            PERL_HASH_SEED=0 hpldir=. perl $script src/sfx.1 > out_seed0.txt
            PERL_HASH_SEED=1 hpldir=. perl $script src/sfx.1 > out_seed1.txt
            # Normalize hash ordering before diffing
            sort out_seed0.txt > norm0.txt
            sort out_seed1.txt > norm1.txt
            diff norm0.txt norm1.txt && echo OK || (echo FAIL; exit 1)
          done

  differential-harness:
    runs-on: ubuntu-latest
    needs: [perl-syntax, reproducibility-test]
```

```
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r requirements.txt
      - run: python harness/run_diff.py --manifest harness/manifest.yaml
```

The **reproducibility-test** job is the operational expression of Prof. Indig's first recommendation: by running each leaf script twice with two different hash seeds and normalising the output, any non-determinism that survives the sort-keys fix becomes immediately visible as a CI failure. The **differential-harness** job implements Prof. Indig's second recommendation — drop-in replacement modules verified against the old ones — and is described in detail in §7. A pre-commit hook running `perltidy` and `perlcritic --stern` will catch the hash-ordering and `each()` issues locally before they reach CI.

7. Differential Harness Scaffolding

The differential harness is the central quality instrument of the port. It runs the original Perl implementation and the new Python port side-by-side on identical inputs and classifies every output difference into one of three buckets: **OLD_BUG** (Perl output is wrong, Python output is correct), **NEW_BUG** (Python output is wrong, Perl output is correct), or **INTENTIONAL_CHANGE** (the difference is documented and intentional, e.g. fixed hash ordering). The classification is driven by a YAML manifest that lists, for every script, the set of intentional changes that the team has signed off on.

Directory layout:

```
emmorph-py/
├── oracle/                    # frozen outputs of the Perl scripts
│   ├── getsfxtags_perl.out    #   run with PERL_HASH_SEED=0
│   ├── encode_perl.out
│   └── ...
├── harness/
│   ├── run_diff.py            # main entry point
│   ├── classify.py             # diff -> OLD_BUG / NEW_BUG / INTENTIONAL
│   ├── normalize.py            # sort hash-like outputs deterministically
│   └── manifest.yaml           # intentional-change manifest
├── pyemmorph/                  # the Python port
│   ├── __init__.py
│   ├── set.py                  # port of pl/generic/set.pl
│   ├── encode.py               # port of pl/generic/encode.pl
│   └── ...
└── tests/
    └── test_differential.py    # pytest wrapper around harness/run_diff.py
```

Manifest format (YAML):

```
# harness/manifest.yaml
scripts:
  getsfxtags:
    perl:   pl/mkavs/getsfxtags.pl
    python: pyemmorph.getsfxtags
    args:   [src/sfx.1]
    env:    { hpldir: "." }
    intentional_changes:
      - kind: hash_ordering_fixed
```

```
        description: "%mcat and %dercat keys now emitted in sorted order"
        normalize: sort_keys
```

run_diff.py interface (sketch):

```
# harness/run_diff.py
import subprocess, json, difflib, yaml
from pathlib import Path

def run_perl(script, args, env):
    env = {**os.environ, 'PERL_HASH_SEED': '0', **env}
    out = subprocess.run(['perl', script, *args],
                         capture_output=True, env=env, text=True)
    return out.stdout

def run_python(module, args):
    # invokes the python port as a CLI module
    out = subprocess.run(['python', '-m', module, *args],
                         capture_output=True, text=True)
    return out.stdout

def classify_diff(perl_out, py_out, manifest_entry):
    p = normalize(perl_out, manifest_entry)
    q = normalize(py_out,  manifest_entry)
    if p == q:
        return []
    diffs = list(difflib.unified_diff(p.splitlines(), q.splitlines(),
                                      'perl', 'python', lineterm=''))
    # Bucket: if perl output is identical to oracle, it's a NEW_BUG.
    # If python output matches a documented intentional_change pattern,
    # it's INTENTIONAL_CHANGE. Otherwise it's an unclassified diff.
    return classify(diffs, manifest_entry)
```

The harness is invoked by the **differential-harness** CI job defined in §6, and is the gate that prevents a regression in the Python port from being merged. Every ported module must achieve a clean differential run — that is, every diff against the Perl oracle must either be empty or be explicitly listed in the manifest — before the module is considered done. This gives the team a binary signal for each ported module: green CI = port is correct, red CI = investigate.

8. Next Steps and Timeline

The following two-to-three-week plan covers the period from 18 October to 8 November 2026. The aim is to deliver, by the next bi-weekly update, a forked repository with a green CI pipeline, the differential harness scaffolding, and the first five ported leaf modules. The choice of leaf modules follows the principle of starting with the smallest, most self-contained scripts — those that have no dependencies on the missing scripts identified in §3.1 — so that the differential harness can be exercised end-to-end from day one.

Week	Task	Deliverable
Week 1 (18–24 Oct)	Fork the repository under `github.com/FarzanehValaee/emmorph-py`; create branch `python-port`; commit the CI skeleton from §6; commit the differential harness scaffolding from §7; generate the Perl oracle outputs with `PERL_HASH_SEED=0` and commit them to `oracle/`.	Fork URL, green CI badge, oracle/ directory with frozen outputs.

Week	Task	Deliverable
Week 2 (25–31 Oct)	Port the five leaf modules — `set.pl`, `bsort.pl`, `lcase.pl`, `sort.pl`, `uniq.pl` — using the differential harness to validate each one. Fix bugs B1, B2, B6 during the port. Add `use FindBin;` patches to the Perl originals so the oracle can be regenerated on modern Perl.	Five ported modules with passing differential tests; B1, B2, B6 fixed.
Week 3 (1–8 Nov)	Port `encode.pl` and `metadict.pl` — the two largest generic modules — and validate against the oracle. Investigate B4 and B5 with Prof. Indig; both require a domain-knowledge decision (data vs script). Begin drafting the paper outline for the venue discussion with Prof. Indig.	encode.py and metadict.py ported; B4 and B5 decision recorded; paper outline draft.

Table 3. Three-week plan from 18 October to 8 November 2026.

Bi-weekly updates will continue at the cadence proposed in the previous email: a short structured note every two weeks, or sooner whenever there is a concrete result — a green CI run, a ported module, a decision needed on a bug. The next update is scheduled for 1 November 2026.

9. References

Novák, A. (2003). Milyen a jó Humor? [What is good Humor like?]. In *Magyar Számítógépes Nyelvészeti Konferencia (MSZNY 2003)*, Szegedi Tudományegyetem, pp. 138–145.

Novák, A. (2014). A New Form of Humor – Mapping Constraint-Based Computational Morphologies to a Finite-State Representation. In *Proceedings of the 9th International Conference on Language Resources and Evaluation (LREC-2014)*, Reykjavík, pp. 1068–1073. ISBN 978-2-9517408-8-4.

Novák, A., Siklósi, B., & Oravecz, Cs. (2016). A New Integrated Open-source Morphological Analyzer for Hungarian. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC 2016)*, Portorož, pp. 1315–1322.

Trón, V. (2004). *Handling agglutinative languages with a constraint-based morphological analyser*. ACL Anthology L04-1259.

Indig, B. (2014). *Vowel harmony, vowel length and palatality in the Humor formalism*. REAL-MTAK 21883.

Perl documentation (2017). *Removal of "." from @INC in Perl 5.26*. perl5260delta.pod.

Creutz, M., & Lagus, K. (2007). Unsupervised models for morpheme segmentation and morphology learning. *ACM Transactions on Speech and Language Processing*, 4(1), article 3.

Soricut, R., & Radu, M. (2026). Dual optimisation in morpheme-aware subword tokenisation. *Expert Systems with Applications*, doi:10.1016/j.eswa.2026.126405.

Kirov, C., Cotterell, R., Sylak-Glassman, J., Walther, G., Vylomova, E., Stark, C., et al. (2018). UniMorph 2.0: Universale Morphological Features in NLP. In *Proceedings of LREC 2018*.